



Introduction on SQL

3.1 SQL Introduction

SQL

- SQL stands for Structured Query Language. It is used for storing and managing data in relational database management system (RDMS).
- It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- SQL allows users to query the database in a number of ways, using English-like statements.

3.2 Rules

Rules:

SQL follows the following rules:

1. Structure query language is not case sensitive. Generally, keywords of SQL are written in uppercase.
2. Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
3. Using the SQL statements, you can perform most of the actions in a database.
4. SQL depends on tuple relational calculus and relational algebra.

3.4 Characteristics of SQL

1. SQL is easy to learn.
2. SQL is used to access data from relational database management systems.
3. SQL can execute queries against the database.
4. SQL is used to describe the data.
5. SQL is used to define the data in the database and manipulate it when needed.
6. SQL is used to create and drop the database and table.
7. SQL is used to create a view, stored procedure, function in a database.
8. SQL allows users to set permissions on tables, procedures, and views.

3.5 Advantages of SQL

There are the following advantages of SQL:

- **High speed**

Using the SQL queries, the user can quickly and efficiently retrieve a large amount of records from a database.

- **No coding needed**

In the standard SQL, it is very easy to manage the database system. It doesn't require a substantial amount of code to manage the database system.

- **Well defined standards**

Long established are used by the SQL databases that are being used by ISO and ANSI.

3.6 SQL Data Types

SQL String Data Type

char(n)	It is a fixed width character string data type. Its size can be up to 8000 characters.
varchar(n)	It is a variable width character string data type. Its size can be up to 8000 characters.
varchar(max)	It is a variable width character string data types. Its size can be up to 1,073,741,824 characters.
text	It is a variable width character string data type. Its size can be up to 2GB of text data.
nchar	It is a fixed width Unicode string data type. Its size can be up to 4000 characters.
nvarchar	It is a variable width Unicode string data type. Its size can be up to 4000 characters.
ntext	It is a variable width Unicode string data type. Its size can be up to 2GB of text data.
binary(n)	It is a fixed width Binary string data type. Its size can be up to 8000 bytes.
varbinary	It is a variable width Binary string data type. Its size can be up to 8000 bytes.
image	It is also a variable width Binary string data type. Its size can be up to 2GB.

3.6.1 Numeric Data Types

SQL Server Numeric Data Types

bit	It is an integer that can be 0, 1 or null.
tinyint	It allows whole numbers from 0 to 255.
Smallint	It allows whole numbers between -32,768 and 32,767.
Int	It allows whole numbers between -2,147,483,648 and 2,147,483,647.
bigint	It allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807.
float(n)	It is used to specify floating precision number data from -1.79E+308 to 1.79E+308. The n parameter indicates whether the field should hold the 4 or 8 bytes. Default value of n is 53.
real	It is a floating precision number data from -3.40E+38 to 3.40E+38.
money	It is used to specify monetary data from -922,337,233,685,477.5808 to 922,337,203,685,477.5807.

3.6.2 Date and Time Data Type

SQL Date and Time Data Type

datetime	It is used to specify date and time combination. It supports range from January 1, 1753, to December 31, 9999 with an accuracy of 3.33 milliseconds.
datetime2	It is used to specify date and time combination. It supports range from January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds
date	It is used to store date only. It supports range from January 1, 0001 to December 31, 9999
time	It stores time only to an accuracy of 100 nanoseconds
timestamp	It stores a unique number when a new row gets created or modified. The time stamp value is based upon an internal clock and does not correspond to real time. Each table may contain only one-time stamp variable.

3.6.3 Other Data Types

SQL Other Data Types

Sql_variant	It is used for various data types except for text, timestamp, and ntext. It stores up to 8000 bytes of data.
XML	It stores XML formatted data. Maximum 2GB.
cursor	It stores a reference to a cursor used for database operations.
table	It stores result set for later processing.
uniqueidentifier	It stores GUID (Globally unique identifier).

3.7 SQL Literals(1)

There are four kinds of literal values supported in SQL. They are : Character string, Bit string, Exact numeric, and Approximate numeric. These are explained as following below.

1.Character strings:-

Character strings are written as a sequence of characters enveloped in single quotes. the only quote character is delineate at intervals a personality string by 2 single quotes. Some example of character strings are :

1. 'My String'
2. 'I love Data Science'
3. '16378'

SQL Literals(2)

2. Bit String:

A bit string is written either as a sequence of 0s and 1s enveloped in single quotes and preceded by the letter 'B' or as a sequence of positional representation system digits enveloped in single quotes and preceded by the letter 'X' some examples are given below :

1. B'10001011'

2. B'1'

3. B'0'

4. X'C 5'

5. X'0'

SQL Literals(3)

3. Exact Numeric:

These literals are written as a signed or unsigned decimal variety probably with mathematical notation. Samples of actual numeric literals are given below :

1. 8
2. 80
3. 80.00
4. 0.8
5. +88.88
6. -88.88

SQL Literals(4)

4. Approximate numeric :

Approximate numeric literals are written as actual numeric literals followed by the letter 'E', followed by a signed or unsigned number. Some example are :

1. 6E6

2. 66.6E6

3. +66E-6

4. 0.66E

5. -6.66E-8

3.8 SQL Commands

Structured Query Language(SQL) as we all know is the database language by the use of which we can perform certain operations on the existing database. We can also use this language to create a database. SQL uses certain commands like Create, Drop, Insert, etc. to carry out the required tasks.

These SQL commands are mainly categorized into four categories as:

DDL – Data Definition Language

DQL – Data Query Language

DML – Data Manipulation Language

DCL – Data Control Language

SQL Commands(DDL)

DDL: Data Definition Language

This includes changes to the structure of the table like creation of table, altering table, deleting a table etc.

All DDL commands are auto-committed. That means it saves all the changes permanently in the database.

List of DDL commands:

- **CREATE:** This command is used to create the database or its objects (like table, index, function, views, store procedure, and triggers).
- **DROP:** This command is used to delete objects from the database.
- **ALTER:** This is used to alter the structure of the database.
- **TRUNCATE:** This is used to remove all records from a table, including all spaces allocated for the records are removed.

SQL Commands(DQL)

- **COMMENT:** This is used to add comments to the data dictionary.
- **RENAME:** This is used to rename an object existing in the database.

DQL (Data Query Language):

DQL statements are used for performing queries on the data within schema objects. The purpose of the DQL Command is to get some schema relation based on the query passed to it. We can define DQL as follows it is a component of SQL statement that allows getting data from the database and imposing order upon it. It includes the **SELECT** statement. This command allows getting the data out of the database to perform operations with it. When a **SELECT** is fired against a table or tables the result is compiled into a further temporary table, which is displayed or perhaps received by the program i.e. a front-end.

SELECT: It is used to retrieve data from the database.

SQL Commands(DDL)

DML: Data Manipulation Language

DML commands are used for manipulating the data stored in the table and not the table itself.

DML commands are not auto-committed. It means changes are not permanent to database, they can be rolled back.

List of DML commands:

- INSERT : It is used to insert data into a table.
- UPDATE: It is used to update existing data within a table.
- DELETE : It is used to delete records from a database table.
- LOCK: Table control concurrency.
- CALL: Call a PL/SQL or JAVA subprogram.
- EXPLAIN PLAN: It describes the access path to data.

SQL Commands(DCL)

DCL: Data Control Language

DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.

List of DCL commands:

- GRANT: This command gives users access privileges to the database.
- REVOKE: This command withdraws the user's access privileges given by using the GRANT command.

SQL Commands(TCL)

TCL: Transaction Control Language

These commands are to keep a check on other commands and their affect on the database. These commands can annul changes made by other commands by rolling the data back to its original state. It can also make any temporary change permanent.

List of TCL commands:

- COMMIT: to permanently save
- ROLLBACK: to undo change in case of any error occurs.
- SAVEPOINT: Sets a savepoint within a transaction.
- SET TRANSACTION: Specify characteristics for the transaction.

3.9 SQL Operators and their Procedures

SQL Operators

Every database administrator and user uses SQL queries for manipulating and accessing the data of database tables and views.

The manipulation and retrieving of the data are performed with the help of reserved words and characters, which are used to perform arithmetic operations, logical operations, comparison operations, compound operations, etc.

What is SQL Operator?

The SQL reserved words and characters are called operators, which are used with a WHERE clause in a SQL query. In SQL, an operator can either be a unary or binary operator. The unary operator uses only one operand for performing the unary operation, whereas the binary operator uses two operands for performing the binary operation.

SQL Operators and their Procedures(conti...)

Syntax of Unary SQL Operator

Operator SQL_Operand

Syntax of Binary SQL Operator

Operand1 SQL_Operator Operand2

Note: SQL operators are used for filtering the table's data by a specific condition in the SQL statement

SQL Operators and their Procedures(conti...)

Types of Operator:

SQL operators are categorized in the following categories:

1. SQL Arithmetic Operators
2. SQL Comparison Operators
3. SQL Logical Operators
4. SQL Set Operators
5. SQL Bit-wise Operators
6. SQL Unary Operators

SQL Operators and their Procedures(conti...)

SQL Arithmetic Operators

The Arithmetic Operators perform the mathematical operation on the numerical data of the SQL tables. These operators perform addition, subtraction, multiplication, and division operations on the numerical operands.

Following are the various arithmetic operators performed on the SQL data:

1. SQL Addition Operator (+)
2. SQL Subtraction Operator (-)
3. SQL Multiplication Operator (*)
4. SQL Division Operator (/)
5. SQL Modulus Operator (%)

SQL Operators and their Procedures(conti...)

SQL Addition Operator (+)

The Addition Operator in SQL performs the addition on the numerical data of the database table. In SQL, we can easily add the numerical values of two columns of the same table by specifying both the column names as the first and second operand. We can also add the numbers to the existing numbers of the specific column.

Syntax of SQL Addition Operator:

```
SELECT operand1 + operand2;
```

SQL Operators and their Procedures(conti...)

This example consists of an **Employee_details** table, which has four columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_Monthlybonus**.

Suppose, we want to add **20,000** to the salary of each employee specified in the table. Then, we have to write the following query in the SQL:

Emp Id	Emp Name	Emp Salary	Emp Monthlybonus
101	Tushar	25000	4000
102	Anuj	30000	200

SQL Operators and their Procedures(conti...)

```
SELECT Emp_Salary + 20000 as Emp_New_Salary FROM  
Employee_details;
```

In this query, we have performed the SQL addition operation on the single column of the given table.

Suppose, we want to add the Salary and monthly bonus columns of the above table, then we have to write the following query in SQL:

```
SELECT Emp_Salary + Emp_Monthlybonus as Emp_Total_Salary  
FROM Employee_details;
```

In this query, we have added two columns with each other of the above table.

SQL Operators and their Procedures(conti...)

SQL Subtraction Operator (-)

The Subtraction Operator in SQL performs the subtraction on the numerical data of the database table. In SQL, we can easily subtract the numerical values of two columns of the same table by specifying both the column names as the first and second operand. We can also subtract the number from the existing number of the specific table column.

Syntax of SQL Subtraction Operator:

```
SELECT operand1 - operand2;
```

SQL Operators and their Procedures(conti...)

- Suppose we want to subtract 5,000 from the salary of each employee given in the **Employee_details** table. Then, we have to write the following query in the SQL:

```
SELECT Emp_Salary -  
5000 as Emp_New_Salary FROM Employee_details;
```

In this query, we have performed the SQL subtraction operation on the single column of the given table.

- If we want to subtract the penalty from the salary of each employee, then we have to write the following query in SQL:

```
SELECT Emp_Salary -  
Penalty as Emp_Total_Salary FROM Employee_details;
```

SQL Operators and their Procedures(conti...)

SQL Multiplication Operator (*)

The Multiplication Operator in SQL performs the Multiplication on the numerical data of the database table. In SQL, we can easily multiply the numerical values of two columns of the same table by specifying both the column names as the first and second operand.

Syntax of SQL Multiplication Operator:

```
SELECT operand1 * operand2;
```

SQL Operators and their Procedures(conti...)

Suppose, we want to double the salary of each employee given in the Employee_details table. Then, we have to write the following query in the SQL:

```
SELECT    Emp_Salary    *    2    as    Emp_New_Salary    FROM  
Employee_details;
```

In this query, we have performed the SQL multiplication operation on the single column of the given table.

If we want to multiply the Emp_Id column to Emp_Salary column of that employee whose Emp_Id is 202, then we have to write the following query in SQL:

```
SELECT Emp_Id * Emp_Salary as Emp_Id * Emp_Salary FROM  
Employee_details WHERE Emp_Id = 202;
```

In this query, we have multiplied the values of two columns by using the WHERE clause.

SQL Operators and their Procedures(conti...)

SQL Division Operator (/)

The Division Operator in SQL divides the operand on the left side by the operand on the right side.

Syntax of SQL Division Operator:

```
SELECT operand1 / operand2;
```

In SQL, we can also divide the numerical values of one column by another column of the same table by specifying both column names as the first and second operand.

We can also perform the division operation on the stored numbers in the column of the SQL table.

SQL Operators and their Procedures(conti...)

Suppose, we want to half the salary of each employee given in the Employee_details table. For this operation, we have to write the following query in the SQL:

```
SELECT    Emp_Salary    /    2    as    Emp_New_Salary    FROM  
Employee_details;
```

In this query, we have performed the SQL division operation on the single column of the given table.

SQL Modulus Operator (%)

The Modulus Operator in SQL provides the remainder when the operand on the left side is divided by the operand on the right side.

Syntax of SQL Modulus Operator:

```
SELECT operand1 % operand2;
```

SQL Operators and their Procedures(conti...)

Number	First operand	Second operand
1	56	4
2	32	8
3	89	9
4	18	10
5	10	5

This example consists of a **Division** table, which has three columns **Number**, **First_operand**, and **Second_operand**.

If we want to get the remainder by dividing the numbers of **First_operand** column by the numbers of **Second_operand** column, then we have to write the following query in SQL:

```
SELECT First_operand % Second_operand as Remainder FROM Employee_details;
```

SQL Operators and their Procedures(conti...)

SQL Comparison Operators

The Comparison Operators in SQL compare two different data of SQL table and check whether they are the same, greater, and lesser. The SQL comparison operators are used with the WHERE clause in the SQL queries

Following are the various comparison operators which are performed on the data stored in the SQL database tables:

1. SQL Equal Operator (=)
2. SQL Not Equal Operator (!=)
3. SQL Greater Than Operator (>)
4. SQL Greater Than Equals to Operator (>=)
5. SQL Less Than Operator (<)\
6. SQL Less Than Equals to Operator (<=)

SQL Operators and their Procedures(conti...)

SQL Equal Operator (=)

This operator is highly used in SQL queries. The Equal Operator in SQL shows only data that matches the specified value in the query.

This operator returns TRUE records from the database table if the value of both operands specified in the query is matched.

Let's understand the below example which explains how to execute Equal Operator in SQL query:

Emp Id	Emp Name	Emp Salary
201	Abhay	30000
202	Ankit	40000
203	Bheem	30000
204	Ram	29000
205	Sumit	30000

SQL Operators and their Procedures(conti...)

This example consists of an Employee_details table, which has three columns Emp_Id, Emp_Name, and Emp_Salary.

Suppose, we want to access all the records of those employees from the Employee_details table whose salary is 30000. Then, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Salary = 30000;
```

In this example, we used the SQL equal operator with WHERE clause for getting the records of those employees whose salary is 30000.

SQL Operators and their Procedures(conti...)

SQL Equal Not Operator (!=)

The Equal Not Operator in SQL shows only those data that do not match the query's specified value.

This operator returns those records or rows from the database views and tables if the value of both operands specified in the query is not matched with each other.

Suppose, we want to access all the records of those employees from the Employee_details table whose salary is not 45000. Then, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Salary != 45000;
```

In this example, we used the SQL equal not operator with WHERE clause for getting the records of those employees whose salary is not 45000.

SQL Operators and their Procedures(conti...)

SQL Greater Than Operator (>)

The Greater Than Operator in SQL shows only those data which are greater than the value of the right-hand operand.

Suppose, we want to access all the records of those employees from the Employee_details table whose employee id is greater than 202. Then, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Id > 202;
```

Here, SQL greater than operator displays the records of those employees from the above table whose Employee Id is greater than 202.

SQL Greater Than Equals to Operator (>=)

The Greater Than Equals to Operator in SQL shows those data from the table which are greater than and equal to the value of the right-hand operand.

SQL Operators and their Procedures(conti...)

Suppose, we want to access all the records of those employees from the Employee_details table whose employee id is greater than and equals to 202. For this, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Id >= 202;
```

Here,'SQL greater than equals to operator' with WHERE clause displays the rows of those employees from the table whose Employee Id is greater than and equals to 202.

SQL Operators and their Procedures(conti...)

SQL Less Than Operator (<)

The Less Than Operator in SQL shows only those data from the database tables which are less than the value of the right-side operand. This comparison operator checks that the left side operand is lesser than the right side operand. If the condition becomes true, then this operator in SQL displays the data which is less than the value of the right-side operand.

Suppose, we want to access all the records of those employees from the Employee_details table whose employee id is less than 204. For this, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Id < 204;
```

Here, SQL less than operator with WHERE clause displays the records of those employees from the above table whose Employee Id is less than 204.

SQL Operators and their Procedures(conti...)

SQL Less Than Equals to Operator (<=)

The Less Than Equals to Operator in SQL shows those data from the table which are lesser and equal to the value of the right-side operand. This comparison operator checks that the left side operand is lesser and equal to the right side operand.

Suppose, we want to access all the records of those employees from the Employee_details table whose employee id is less and equals 203. For this, we have to write the following query in the SQL database:

```
SELECT * FROM Employee_details WHERE Emp_Id <= 203;
```

Here, SQL less than equals to the operator with WHERE clause displays the rows of those employees from the table whose Employee Id is less than and equals 202.

SQL Operators and their Procedures(conti...)

SQL Logical Operators

The **Logical Operators** in SQL perform the Boolean operations, which give two results **True and False**. These operators provide **True** value if both operands match the logical condition.

Following are the various logical operators which are performed on the data stored in the SQL database tables:

- 1.SQL ALL operator
- 2.SQL AND operator
- 3.SQL OR operator
- 4.SQL BETWEEN operator
- 5.SQL IN operator
- 6.SQL NOT operator
- 7.SQL ANY operator
- 8.SQL LIKE operator

SQL Operators and their Procedures(conti...)

SQL ALL Operator

The ALL operator in SQL compares the specified value to all the values of a column from the sub-query in the SQL database.

This operator is always used with the following statement:

SELECT,
HAVING, and
WHERE.

Syntax of ALL operator:

```
SELECT column_Name1, ....., column_NameN FROM table_Name WHERE  
column Comparison_operator
```

Let's understand the below example which explains how to execute ALL logical operators in SQL query:

If we want to access the employee id and employee names of those employees from the table whose salaries are greater than the salary of

SQL Operators and their Procedures(conti...)

employees who lives in Jaipur city, then we have to type the following query in SQL.

Here

```
SELECT Emp_Id, Emp_Name FROM Employee_details WHERE  
Emp_Salary > ALL ( SELECT Emp_Salary FROM Employee_det  
ails WHERE Emp_City = Jaipur)
```

, we used the **SQL ALL operator** with greater than the operator.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Gurgaon
202	Ankit	45000	Delhi
203	Bheem	30000	Jaipur
204	Ram	29000	Mumbai
205	Sumit	40000	Kolkata

SQL Operators and their Procedures(conti...)

SQL AND Operator

The **AND operator** in SQL would show the record from the database table if all the conditions separated by the AND operator evaluated to True. It is also known as the conjunctive operator and is used with the WHERE clause.

Syntax of AND operator:

```
SELECT column1, ..., columnN FROM table_Name WHERE condition1 AND condition2 AND condition3
```

Suppose, we want to access all the records of those employees from the **Employee_details** table whose salary is 25000 and the city is Delhi. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Salary = 25000  
AND Emp_City = 'Delhi';
```

Here, **SQL AND operator** with WHERE clause shows the record of employees whose salary is 25000 and the city is Delhi.

SQL Operators and their Procedures(conti...)

SQL OR Operator

The OR operator in SQL shows the record from the table if any of the conditions separated by the OR operator evaluates to True. It is also known as the conjunctive operator and is used with the WHERE clause.

Syntax of OR operator:

```
SELECT column1, ....., columnN FROM table_Name WHERE condition1 OR condition2 OR condition3 OR .
```

If we want to access all the records of those employees from the **Employee_details** table whose salary is 25000 or the city is Delhi. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Salary = 25000 OR Emp_City = 'Delhi';
```

Here, **SQL OR operator** with WHERE clause shows the record of employees whose salary is 25000 or the city is Delhi.

SQL Operators and their Procedures(conti...)

SQL BETWEEN Operator

The BETWEEN operator in SQL shows the record within the range mentioned in the SQL query. This operator operates on the numbers, characters, and date/time operands.

Suppose, we want to access all the information of those employees from the Employee_details table who is having salaries between 20000 and 40000. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Salary BETWEEN 30000 AND 45000;
```

Here, we used the SQL BETWEEN operator with the Emp_Salary field.

SQL IN Operator

The **IN operator** in SQL allows database users to specify two or more values in a WHERE clause. This logical operator minimizes the requirement of multiple OR conditions.

This operator makes the query easier to learn and understand. This operator returns those rows whose values match with any value of the given list.

SQL Operators and their Procedures(conti...)

Suppose, we want to show all the information of those employees from the **Employee_details** table whose **Employee Id** is 202, 204, and 205. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Id IN (202, 204, 205);
```

Here, we used the **SQL IN operator** with the Emp_Id column.

Suppose, we want to show all the information of those employees from the **Employee_details** table whose **Employee Id** is not equal to 202 and 205. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Id NOT IN (202, 205);
```

Here, we used the **SQL NOT IN operator** with the Emp_Id column.

SQL Operators and their Procedures(conti...)

SQL NOT Operator

The **NOT operator** in SQL shows the record from the table if the condition evaluates to false. It is always used with the WHERE clause.

Syntax of NOT operator:

SELECT column1, column2 ..., columnN FROM table_Name WHERE NOT condition;

Suppose, we want to show all the information of those employees from the **Employee_details** table whose City is not Delhi. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE NOT Emp_City = 'Delhi' ;
```

In this example, we used the **SQL NOT operator** with the Emp_City column.

Suppose, we want to show all the information of those employees from the **Employee_details** table whose City is not Delhi and Chandigarh. For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE NOT Emp_City = 'Delhi' AND NOT  
Emp_City = 'Chandigarh';
```

- In this example, we used the **SQL NOT operator** with the Emp_City column.

SQL Operators and their Procedures(conti...)

ANY/SOME

The ANY operator returns TRUE if any of the subquery values meet the specified condition. In the below example, we are filtering all products which have any record in the orders table. The SOME operator achieves the same result.

```
SELECT product_name FROM products WHERE product_id > ANY (SELECT product_id FROM Orders;
```

SQL LIKE Operator

The **LIKE operator** in SQL shows those records from the table which match with the given pattern specified in the sub-query.

The percentage (%) sign is a wildcard which is used in conjunction with this logical operator.

This operator is used in the WHERE clause with the following three statements:

- 1.SELECT statement
- 2.UPDATE statement
- 3.DELETE statement

SQL Operators and their Procedures(conti...)

This example consists of an **Employee_details** table, which has four columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Sanjay	25000	Delhi
202	Ajay	45000	Chandigarh
203	Saket	30000	Delhi
204	Abhay	25000	Delhi
205	Sumit	40000	Kolkata

SQL Operators and their Procedures(conti...)

If we want to show all the information of those employees from the **Employee_details** whose name starts with "s". For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Name LIKE 's%';
```

In this example, we used the SQL LIKE operator with **Emp_Name** column because we want to access the record of those employees whose name starts with s.

If we want to show all the information of those employees from the **Employee_details** whose name ends with "y". For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Name LIKE '%y';
```

If we want to show all the information of those employees from the **Employee_details** whose name starts with "S" and ends with "y". For this, we have to write the following query in SQL:

```
SELECT * FROM Employee_details WHERE Emp_Name LIKE 'S%y';
```

SQL Operators and their Procedures(conti...)

SQL Bitwise Operators

The **Bitwise Operators** in SQL perform the bit operations on the Integer values. To understand the performance of Bitwise operators, you just knew the basics of Boolean algebra.

Following are the two important logical operators which are performed on the data stored in the SQL database tables:

1.Bitwise AND (&)

2.Bitwise OR(|)

Bitwise AND (&)

The Bitwise AND operator performs the logical AND operation on the given Integer values. This operator checks each bit of a value with the corresponding bit of another value.

Syntax of Bitwise AND Operator

1.SELECT column1 & column2 & & columnN FROM table_Name [WHERE conditions] ;

Let's understand the below example which explains how to execute Bitwise AND operator on the data of SQL table:

This example consists of the following table, which has two columns. Each column holds numerical values.

SQL Operators and their Procedures(conti...)

Column1	Column2
1	1
2	5
3	4
4	2
5	3

When we use the Bitwise AND operator in SQL, then SQL converts the values of both columns in binary format, and the AND operation is performed on the converted bits. After that, SQL converts the resultant bits into user understandable format, i.e., decimal format.

Suppose, we want to perform the Bitwise AND operator between both the columns of the above table. For this, we have to write the following query in SQL:

```
SELECT Column1 & Column2 From TABLE_AND ;
```

SQL Operators and their Procedures(conti...)

Bitwise OR (|)

The Bitwise OR operator performs the logical OR operation on the given Integer values. This operator checks each bit of a value with the corresponding bit of another value.

Syntax of Bitwise OR Operator

```
SELECT column1 | column2 | .... | columnN FROM table_Name [
WHERE conditions] ;
```

When we used the Bitwise OR operator in SQL, then SQL converts the values of both columns in binary format, and the OR operation is performed on the binary bits. After that, SQL converts the resultant binary bits into user understandable format, i.e., decimal format.

Suppose, we want to perform the Bitwise OR operator between both the columns of the above table. For this, we have to write the following query in SQL:

```
SELECT Column1 | Column2 From TABLE_OR ;
```

3.10 Tables in SQL

SQL Table

Table is a collection of data, organized in terms of rows and columns. In DBMS term, table is known as relation and row as tuple.

Note: A table has a specified number of columns but can have any number of rows.

Table is the simple form of data storage. A table is also considered as a convenient representation of relations.

Let's see an example of an employee table:

EMP_NAME	ADDRESS	SALARY
Ankit	Lucknow	15000
Raman	Allahabad	18000
Mike	New York	20000

Tables in SQL(conti..)

In the above table, "Employee" is the table name, "EMP_NAME", "ADDRESS" and "SALARY" are the column names. The combination of data of multiple columns forms a row e.g. "Ankit", "Lucknow" and 15000 are the data of one row.

SQL TABLE Variable

The **SQL Table variable** is used to create, modify, rename, copy and delete tables. It is a variable where we temporary store records and results. This is same like temp table but in the case of temp table we need to explicitly drop it.

Table variables are used to store a set of records. So declaration syntax generally looks like CREATE TABLE syntax.

create table "tablename"

("column1" "data type",

"column2" "data type",

...

"columnN" "data type");

Tables in SQL(conti..)

When a transaction rolled back the data associated with table variable is not rolled back. A table variable generally uses lesser resources than a temporary variable.

Table variable cannot be used as an input or an output parameter.

SQL CREATE TABLE

SQL CREATE TABLE statement is used to create table in a database.

If you want to create a table, you should name the table and define its column and each column's data type.

Let's see the simple syntax to create the table.

```
create table "tablename"  
("column1" "data type",  
"column2" "data type",  
...  
"columnN" "data type");
```

Tables in SQL(conti..)

Let us take an example to create a STUDENTS table with ID as primary key and NOT NULL are the constraint showing that these fields cannot be NULL while creating records in the table.

```
SQL> CREATE TABLE STUDENTS (
```

```
ID INT NOT NULL,
```

```
NAME VARCHAR (20) NOT NULL,
```

```
AGE INT NOT NULL,
```

```
ADDRESS CHAR (25),
```

```
PRIMARY KEY (ID) );
```

SQL CREATE TABLE Example in Microsoft SQLServer

```
CREATE TABLE Employee
```

```
( EmployeeID int,
```

```
FirstName varchar(255),
```

```
LastName varchar(255),
```

```
Email varchar(255),
```

```
AddressLine varchar(255),
```

```
City varchar(255) );
```

Tables in SQL(conti..)

SQL DROP TABLE

A SQL DROP TABLE statement is used to delete a table definition and all data from a table.

This is very important to know that once a table is deleted all the information available in the table is lost forever, so we have to be very careful when using this command.

Let's see the syntax to drop the table from the database.

DROP TABLE "table_name";

SQL DELETE TABLE

The DELETE statement is used to delete rows from a table. If you want to remove a specific row from a table you should use WHERE condition.

DELETE FROM table_name [**WHERE** condition];

But if you do not specify the WHERE condition it will remove all the rows from the table.

DELETE FROM table_name;

Tables in SQL(conti..)

There are some more terms similar to DELETE statement like as DROP statement and TRUNCATE statement but they are not exactly same there are some differences between them.

Difference between DELETE and TRUNCATE statements

There is a slight difference b/w delete and truncate statement. The **DELETE statement** only deletes the rows from the table based on the condition defined by WHERE clause or delete all the rows from the table when condition is not specified.

But it does not free the space containing by the table.

The **TRUNCATE statement**: it is used to delete all the rows from the table **and free the containing space**.

Tables in SQL(conti..)

Difference b/w DROP and TRUNCATE statements

When you use the drop statement it deletes the table's row together with the table's definition so all the relationships of that table with other tables will no longer be valid.

When you drop a table:

- Table structure will be dropped.
- Relationship will be dropped.
- Integrity constraints will be dropped.
- Access privileges will also be dropped.
- On the other hand when we **TRUNCATE** a table, the table structure remains the same, so you will not face any of the above problems.

Tables in SQL(conti..)

SQL RENAME TABLE

In some situations, database administrators and users want to change the name of the table in the SQL database because they want to give a more relevant name to the table.

Any database user can easily change the name by using the RENAME TABLE and ALTER TABLE statement in Structured Query Language.

The RENAME TABLE and ALTER TABLE syntax help in changing the name of the table.

Syntax of RENAME statement in SQL

```
RENAME old_table _name To new_table_name ;
```

SQL COPY TABLE

If you want to copy the data of one SQL table into another SQL table in the same SQL server, then it is possible by using the SELECT INTO statement in SQL.

Tables in SQL(conti..)

The SELECT INTO statement in Structured Query Language copies the content from one existing table into the new table. SQL creates the new table by using the structure of the existing table.

Syntax of SELECT INTO statement in SQL

SELECT * INTO New_table_name FROM old_table_name;

SQL ALTER TABLE

The ALTER TABLE statement in Structured Query Language allows you to add, modify, and delete columns of an existing table. This statement also allows database users to add and remove various SQL constraints on the existing tables.

Any user can also change the name of the table using this statement.

ALTER TABLE ADD Column statement in SQL

In many situations, you may require to add the columns in the existing table. Instead of creating a whole table or database again you can easily add single and multiple columns using the ADD keyword.

Tables in SQL(conti..)

Syntax of ALTER TABLE ADD Column statement in SQL

ALTER TABLE table_name ADD column_name column-definition;

The above syntax only allows you to add a single column to the existing table. If you want to add more than one column to the table in a single SQL statement, then use the following syntax:

ALTER TABLE table_name

ADD (column_Name1 column-definition,

column_Name2 column-definition,

.....

column_NameN column-definition);

3.11 Views in SQL

Views in SQL

Views in SQL are considered as a virtual table. A view also contains rows and columns.

To create the view, we can select the fields from one or more tables present in the database.

A view can either have specific rows based on certain condition or all the rows of a table.

Sample table:

Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Views in SQL(conti..)

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

Creating view

A view can be created using the **CREATE VIEW** statement. We can create a view from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS  
SELECT column1, column2.....  
FROM table_name  
WHERE condition;
```

Views in SQL

2. Creating View from a single table

In this example, we create a View named DetailsView from the table Student_Detail.

Query:

```
CREATE VIEW DetailsView AS  
SELECT NAME, ADDRESS  
FROM Student_Details  
WHERE STU_ID < 4;
```

Just like table query, we can query the view to view the data.

```
SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

Views in SQL

3. Creating View from multiple tables

View from multiple tables can be created by simply include multiple tables in the SELECT statement.

In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

Query:

```
CREATE VIEW MarksView AS
SELECT Student_Detail.NAME, Student_Detail.ADDRESS, Student_Marks.MA
RKS
FROM Student_Detail, Student_Mark
WHERE Student_Detail.NAME = Student_Marks.NAME;
To display data of View MarksView:
SELECT * FROM MarksView;
```

Views in SQL

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

Views in SQL

4. Deleting View

A view can be deleted using the Drop View statement.

Syntax

```
DROP VIEW view_name;
```

Example:

If we want to delete the View **MarksView**, we can do this as:

```
DROP VIEW MarksView;
```

3.12 SQL Index

SQL Index

Indexes are special lookup tables. It is used to retrieve data from the database very fast.

An Index is used to speed up select queries and where clauses. But it slows down the data input with insert and update statements. Indexes can be created or dropped without affecting the data.

An index in a database is just like an index in the back of a book.

For example: When you reference all pages in a book that discusses a certain topic, you first have to refer to the index, which alphabetically lists all the topics and then referred to one or more specific page numbers.

Create Index statement

It is used to create an index on a table. It allows duplicate value.

Syntax

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

Example

```
CREATE INDEX idx_name ON Persons (LastName, FirstName);
```

SQL Index(conti...)

2. Unique Index statement

It is used to create a unique index on a table. It does not allow duplicate values.

Syntax

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

Example

```
CREATE UNIQUE INDEX websites_idx  
ON websites (site_name);
```

3. Drop Index Statement

It is used to delete an index in a table.

Syntax

```
DROP INDEX index_name;
```

Example

```
DROP INDEX websites_idx;
```

3.13 SQL Subqueries

SQL Sub Query

A Subquery is a query within another SQL query and embedded within the WHERE clause.

Important Rule:

- A subquery can be placed in a number of SQL clauses like WHERE clause, FROM clause, HAVING clause.
- You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery is a query within another query. The outer query is known as the main query, and the inner query is known as a subquery.
- Subqueries are on the right side of the comparison operator.
- A subquery is enclosed in parentheses.
- In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

Subqueries with the Select Statement

1. Subqueries with the Select Statement

SQL subqueries are most frequently used with the Select statement.

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name expression operator  
( SELECT column_name from table_name WHERE ... );
```

Subqueries with the Select Statement(Example)

Example

Consider the EMPLOYEE table have the following records:

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
4	Alina	29	UK	6500.00
5	Kathrin	34	Bangalore	8500.00
6	Harry	42	China	4500.00
7	Jackson	25	Mizoram	10000.00

SQL Subqueries Example

The subquery with a SELECT statement will be:

```
SELECT *  
FROM EMPLOYEE  
WHERE ID IN (SELECT ID  
FROM EMPLOYEE  
WHERE SALARY > 4500);
```

This would produce the following result:

ID	NAME	AGE	ADDRESS	SALARY
4	Alina	29	UK	6500.00
5	Kathrin	34	Bangalore	8500.00
7	Jackson	25	Mizoram	10000.00

Subqueries with the INSERT Statement

2. Subqueries with the INSERT Statement

SQL subquery can also be used with the Insert statement. In the insert statement, data returned from the subquery is used to insert into another table.

In the subquery, the selected data can be modified with any of the character, date functions.

Syntax:

```
INSERT INTO table_name (column1, column2, column3....)
```

```
SELECT *
```

```
FROM table_name
```

```
WHERE VALUE OPERATOR
```

Subqueries with the UPDATE Statement

3. Subqueries with the UPDATE Statement

The subquery of SQL can be used in conjunction with the Update statement. When a subquery is used with the Update statement, then either single or multiple columns in a table can be updated.

Syntax

UPDATE table

SET column_name = new_value

WHERE VALUE OPERATOR

(SELECT COLUMN_NAME

FROM TABLE_NAME

WHERE condition);

Subqueries with the DELETE Statement

4. Subqueries with the DELETE Statement

The subquery of SQL can be used in conjunction with the Delete statement just like any other statements mentioned above.

Syntax

```
DELETE FROM TABLE_NAME  
WHERE VALUE OPERATOR  
      (SELECT COLUMN_NAME  
       FROM TABLE_NAME  
       WHERE condition);
```

Subqueries with the DELETE Statement (Example)

Example

Let's assume we have an EMPLOYEE_BKP table available which is backup of EMPLOYEE table. The given example deletes the records from the EMPLOYEE table for all EMPLOYEE whose AGE is greater than or equal to 29.

1.DELETE FROM EMPLOYEE

2. WHERE AGE IN (SELECT AGE FROM EMPLOYEE_BKP

3. WHERE AGE >= 29);

This would impact three rows, and finally, the EMPLOYEE table would have the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
7	Jackson	25	Mizoram	10000.00

Aggregate Functions

➤ Aggregate Functions

➤ Types of Aggregate Functions

- Count Function
- SUM Function
- AVG Function
- MAX Function
- MIN Function

Aggregate Functions

An aggregate function in SQL performs a calculation on multiple values and returns a single value. [SQL](#) provides many aggregate functions that include avg, count, sum, min, max, etc. An aggregate function ignores NULL values when it performs the calculation, except for the count function.

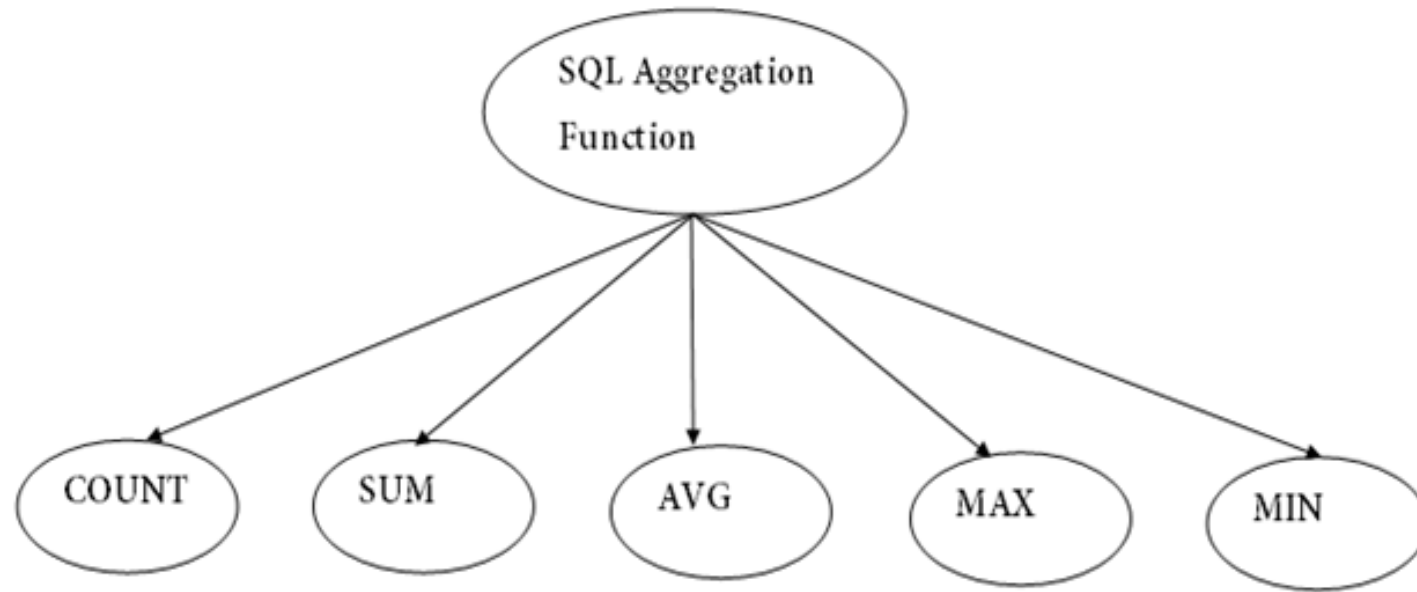
SQL Aggregate Functions

SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.

It is also used to summarize the data.

4.1 Types of Aggregate Functions

Types of SQL Aggregation Function



4.1.1 Count Function

COUNT FUNCTION

COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.

COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax:

COUNT(*)

or

COUNT([ALL | DISTINCT] expression)

4.1.2 Count Function(Example)

Sample table:
PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50
Item10	Com3	4	30	120

Count Function(Example)

Example: COUNT()

```
SELECT COUNT(*) FROM PRODUCT_MAST;
```

Output:

10

Example: COUNT with WHERE

```
SELECT COUNT(*) FROM PRODUCT_MAST; WHERE RATE>=20;
```

Output:

7

Example: COUNT() with DISTINCT

```
SELECT COUNT(DISTINCT COMPANY) FROM PRODUCT_MAST;
```

Output:

3

Example: COUNT() with GROUP BY

```
SELECT COMPANY, COUNT(*) FROM PRODUCT_MAST GROUP BY COMPANY;
```

Output:

Com1

Com2

Com3

4.1.3 SUM Function

Example: COUNT() with HAVING
SELECT COMPANY, COUNT(*)
FROM PRODUCT_MAST
GROUP BY COMPANY
HAVING COUNT(*)>2;

Output:

Com1 5
Com2 3

2. SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax:SUM()

or

SUM([ALL|DISTINCT] expression)

Example: SUM()

SELECT SUM(COST)
FROM PRODUCT_MAST;

SUM Function(Example)

Example: SUM()

```
SELECT SUM(COST)
FROM PRODUCT_MAST;
```

Output:

670

Example: SUM() with WHERE

```
SELECT SUM(COST)
FROM PRODUCT_MAST
WHERE QTY>3;
```

Output:

320

Example: SUM() with GROUP BY

```
SELECT SUM(COST)
FROM PRODUCT_MAST
WHERE QTY>3
GROUP BY COMPANY;
```

Output:

Com1 150

Com3 170

4.1.4 AVG Function

Example: SUM() with HAVING
SELECT COMPANY, SUM(COST)
FROM PRODUCT_MAST
GROUP BY COMPANY
HAVING SUM(COST)>=170;

Output:

Com1 335
Com3 170

3. AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

AVG()

or

AVG([ALL | DISTINCT] expression)

Example:

SELECT AVG(COST)
FROM PRODUCT_MAST;

Output:

67.00

4.1.5 MAX Function

4. MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

MAX()

or

MAX([ALL | DISTINCT] expression)

Example:

```
SELECT MAX(RATE)
FROM PRODUCT_MAST;
30
```

4.1.6 MIN Function

5. MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

- 1.MIN()
- 2.or
- 3.MIN([ALL| DISTINCT] expression)

Example:

- 1.SELECT MIN(RATE)
- 2.FROM PRODUCT_MAST;

Output:

10

Insert, Update and Delete Operations in SQL

The INSERT INTO statement is used to insert new records in a table.

It is possible to write the INSERT INTO statement in two ways.

Syntax

The first way specifies both the column names and the values to be inserted.

If you are adding values for all the columns of the table, then no need to specify the column names in the SQL query. However, make sure that the order of the values is in the same order as the columns in the table.

INSERT INTO table_name (column1, column2, column3, ...)

VALUES (value1, value2, value3, ...);

'2nd way

INSERT INTO table_name

VALUES (value1, value2, value3, ...);

5.1 INSERT operation Example

Example

Insert value in a 1st way. The column names are used here

```
INSERT INTO Employee (EmpId,LastName,FirstName,ADDRESS,  
City)
```

```
VALUES (1, 'XYZ', 'ABC', 'India', 'Mumbai' );
```

```
INSERT INTO Employee (EmpId,LastName,FirstName,ADDRESS,  
City)
```

```
VALUES (2, 'X', 'A', 'India', 'Pune' );
```

Insert value in a 2nd way.

```
INSERT INTO Employee
```

```
VALUES (3, 'XYZ', 'ABC', 'India', 'Mumbai' );
```

5.2 UPDATE Operation

Update Table :

The UPDATE statement is used to modify the existing records in a table.

Syntax

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

Example

UPDATE Employee

SET FirstName= 'KS', City= 'Pune'

WHERE EmpId= 1;

5.3 Delete operation

If the above query is executed then for EmpId= 1, "Firstname" and "City" column data will be updated.

Update Multiple Rows

It is the WHERE clause that determines how many records will be updated.

UPDATE Employee

SET City='Pune'

Delete in SQL

The DELETE statement is used to delete existing records in a table for a particular Record.

Syntax

DELETE FROM table_name **WHERE** condition;

Delete Operation(Example)

Example

DELETE FROM Employee WHERE EmpId=1;

In Employee table EmpId = 1 record gets deleted.

Delete All Records

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact,

DELETE FROM table_name;

DELETE From Employee ;

When the above query is executed, only table Data gets deleted.

6.SQL Joins

➤ JOINS in SQL

➤ Types of SQL joins

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN

6.1 SQL Join

As the name shows, JOIN means to combine something. In case of SQL, JOIN means "to combine two or more tables".

In SQL, JOIN clause is used to combine the records from two or more tables in a database.

Types of SQL JOIN

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN

SQL Joins

Sample Table
EMPLOYEE

EMP_ID	EMP_NAME	CITY	SALARY	AGE
1	Angelina	Chicago	200000	30
2	Robert	Austin	300000	26
3	Christian	Denver	100000	42
4	Kristen	Washington	500000	29
5	Russell	Los angeles	200000	36
6	Marry	Canada	600000	48

PROJECT

PROJECT_NO	EMP_ID	DEPARTMENT
101	1	Testing
102	2	Development
103	3	Designing
104	4	Development

6.2.1 INNER Join

1. INNER JOIN

In SQL, INNER JOIN selects records that have matching values in both tables as long as the condition is satisfied. It returns the combination of all rows from both the tables where the condition satisfies.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1  
INNER JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Query

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT  
FROM EMPLOYEE  
INNER JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

6.2.2 LEFT Join

Output:

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development

2. LEFT JOIN

The SQL left join returns all the values from left table and the matching values from the right table. If there is no matching join value, it will return NULL.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1  
LEFT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

LEFT Join(Example)

Query

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT  
FROM EMPLOYEE  
LEFT JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Output:

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development
Russell	NULL
Marry	NULL

6.2.3 RIGHT Join with Example

3. RIGHT JOIN

In SQL, RIGHT JOIN returns all the values from the values from the rows of right table and the matched values from the left table. If there is no matching in both tables, it will return NULL.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1  
RIGHT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Query

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT  
FROM EMPLOYEE  
RIGHT JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

6.2.4 Full Join

Output:

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development

4. FULL JOIN

In SQL, FULL JOIN is the result of a combination of both left and right outer join. Join tables have all the records from both tables. It puts NULL on the place of matches not found.

Full Join Syntax

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,.  
...  
FROM table1  
FULL JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Query

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTME  
NT  
FROM EMPLOYEE  
FULL JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Full Join Example

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development
Russell	NULL
Marry	NULL

7. SQL Union Operator

SQL Union Operator

The SQL Union Operator combines the result of two or more SELECT statements and provides the single output.

The data type and the number of columns must be the same for each SELECT statement used with the UNION operator. This operator does not show the duplicate records in the output table.

Syntax of UNION Set operator:

```
SELECT column1, column2 ....., columnN FROM table_Name1 [W  
HERE conditions]
```

```
UNION
```

```
SELECT column1, column2 ....., columnN FROM table_Name2 [W  
HERE conditions];
```

SQL Intersect Operator

SQL Intersect Operator

The SQL Intersect Operator shows the common record from two or more SELECT statements. The data type and the number of columns must be the same for each SELECT statement used with the INTERSECT operator.

Syntax of INTERSECT Set operator:

```
SELECT column1, column2 ...., columnN FROM table_Name1  
[WHERE conditions]
```

```
INTERSECT
```

```
SELECT column1, column2 ...., columnN FROM table_Name2  
[WHERE conditions];
```

SQL Minus Operator

SQL Minus Operator

The SQL Minus Operator combines the result of two or more SELECT statements and shows only the results from the first data set.

Syntax of MINUS operator:

```
SELECT column1, column2 ...., columnN FROM First_table  
name [WHERE conditions]
```

MINUS

```
SELECT column1, column2 ...., columnN FROM Second_ta  
blename [WHERE conditions];
```

8. Cursors

- Cursor Definition
- Life cycle of Cursor

8.1 Cursors Definition

Cursor is a Temporary Memory or Temporary Workstation. It is Allocated by Database Server at the Time of Performing DML(Data Manipulation Language) operations on Table by User. Cursors are used to store Database Tables. There are 2 types of Cursors: Implicit Cursors, and Explicit Cursors. These are explained as following below.

1. Implicit Cursors:

Implicit Cursors are also known as Default Cursors of SQL SERVER. These Cursors are allocated by SQL SERVER when the user performs DML operations.

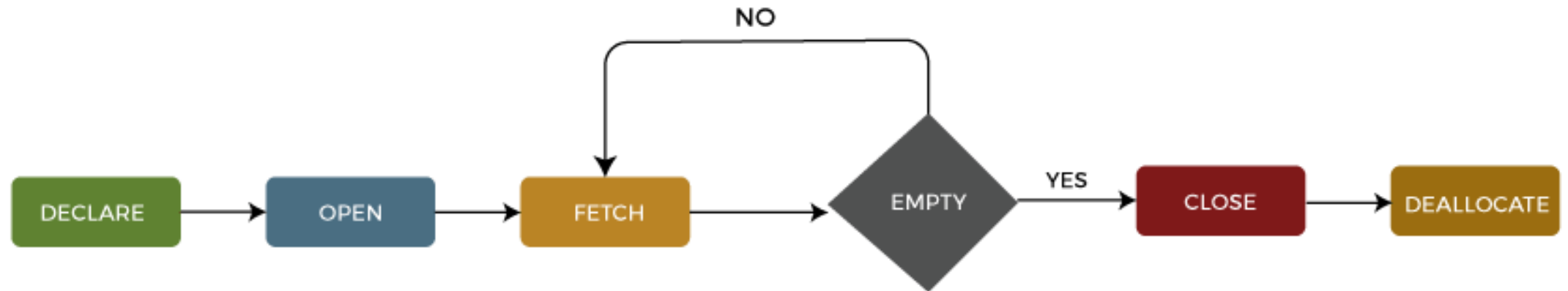
2. Explicit Cursors:

Explicit Cursors are Created by Users whenever the user requires them. Explicit Cursors are used for Fetching data from Table in Row-By-Row Manner.

8.2 Life Cycle of Cursor

Life Cycle of the cursor

We can describe the life cycle of a cursor into the five different sections as follows:



Cursors(conti...)

How to create Explicit Cursor:

1.Declare Cursor Object.

Syntax : DECLARE cursor_name CURSOR FOR SELECT * FROM table_name

```
DECLARE s1 CURSOR FOR SELECT * FROM studDetails
```

2.Open Cursor Connection.

Syntax : OPEN cursor_connection
OPEN s1

3.Fetch Data from cursor.

There are total 6 methods to access data from cursor. They are as follows :

FIRST is used to fetch only the first row from cursor table.

LAST is used to fetch only last row from cursor table.

NEXT is used to fetch data in forward direction from cursor table.

PRIOR is used to fetch data in backward direction from cursor table.

Cursors(conti...)

3.ABSOLUTE n is used to fetch the exact n^{th} row from cursor table.

RELATIVE n is used to fetch the data in incremental way as well as decremental way.

Syntax : FETCH NEXT/FIRST/LAST/PRIOR/ABSOLUTE
n/RELATIVE n FROM cursor_name

FETCH FIRST FROM s1

FETCH LAST FROM s1

FETCH NEXT FROM s1

FETCH PRIOR FROM s1

FETCH ABSOLUTE 7 FROM s1

FETCH RELATIVE -2 FROM s1

4.Close cursor connection.

Syntax : CLOSE cursor_name
CLOSE s1

5.Deallocate cursor memory.

Syntax : DEALLOCATE cursor_name
DEALLOCATE s1

9. Trigger

A SQL trigger is a database object which fires when an event occurs in a database. We can execute a SQL query that will "do something" in a database when a change occurs on a database table such as a record is inserted or updated or deleted. For example, a trigger can be set on a record insert in a database table.

Types of Triggers

- There are two types of triggers:

1.DDL Trigger

2.DML Trigger

DDL Trigger

DDL Triggers

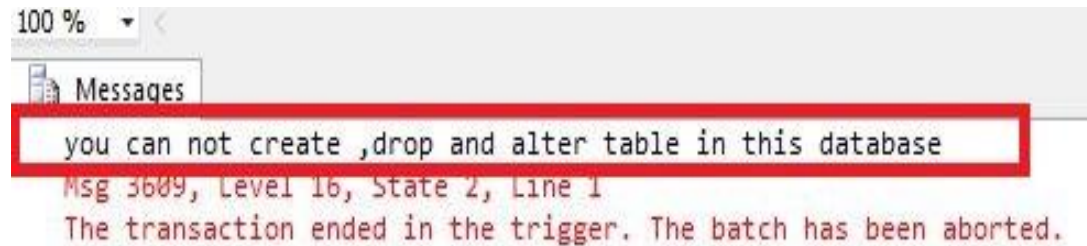
The DDL triggers are fired in response to DDL (Data Definition Language) command events that start with Create, Alter and Drop, such as Create_table, Create_view, drop_table, Drop_view and Alter_table.

Code of a DDL Trigger

```
create trigger saftey
on database
for
create_table,alter_table,drop_table
as
print'you can not create ,drop and alter table in this database'
rollback;
```

DDL Trigger(conti...)

When we create, alter or drop any table in a database then the following message appears:



DML Trigger(conti...)

DML Triggers

The DML triggers are fired in response to DML (Data Manipulation Language) command events that start with Insert, Update, and Delete. Like insert_table, Update_view and Delete_table.

```
create trigger deep
```

```
on emp
```

```
for
```

```
insert,update,delete
```

```
as
```

```
print'you can not insert,update and delete this table i'
```

```
rollback;
```

DML Trigger(conti...)

When we insert, update or delete in a table in a database then the following message appears



DML Trigger(conti...)

There are two types of DML triggers

AFTER Triggers

AFTER triggers are executed after the action of an INSERT, UPDATE, or DELETE statement.

```
create trigger insertt  
on emp  
after insert  
as  
begin  
insert into empstatus values('active')  
end
```

DML Trigger(conti...)

100 %

Results Messages

	status
1	active
2	active

DML Trigger(conti...)

INSTEAD Of Triggers

It will tell the database engine to execute the trigger instead of executing the statement. For example an insert trigger executes when an event occurs instead of the statement that would insert the values in the table .

```
CREATE TRIGGER instoftr  
ON v11  
INSTEAD OF INSERT  
AS  
BEGIN  
INSERT INTO emp  
SELECT l.id, l.names
```

DML Trigger(conti...)

```
FROM INSERTED I  
INSERT INTO emp1values  
SELECT I.id1, I.name1  
FROM INSERTED I  
END  
SQL
```

When we insert data into a view by the following query then it inserts values in both tables :

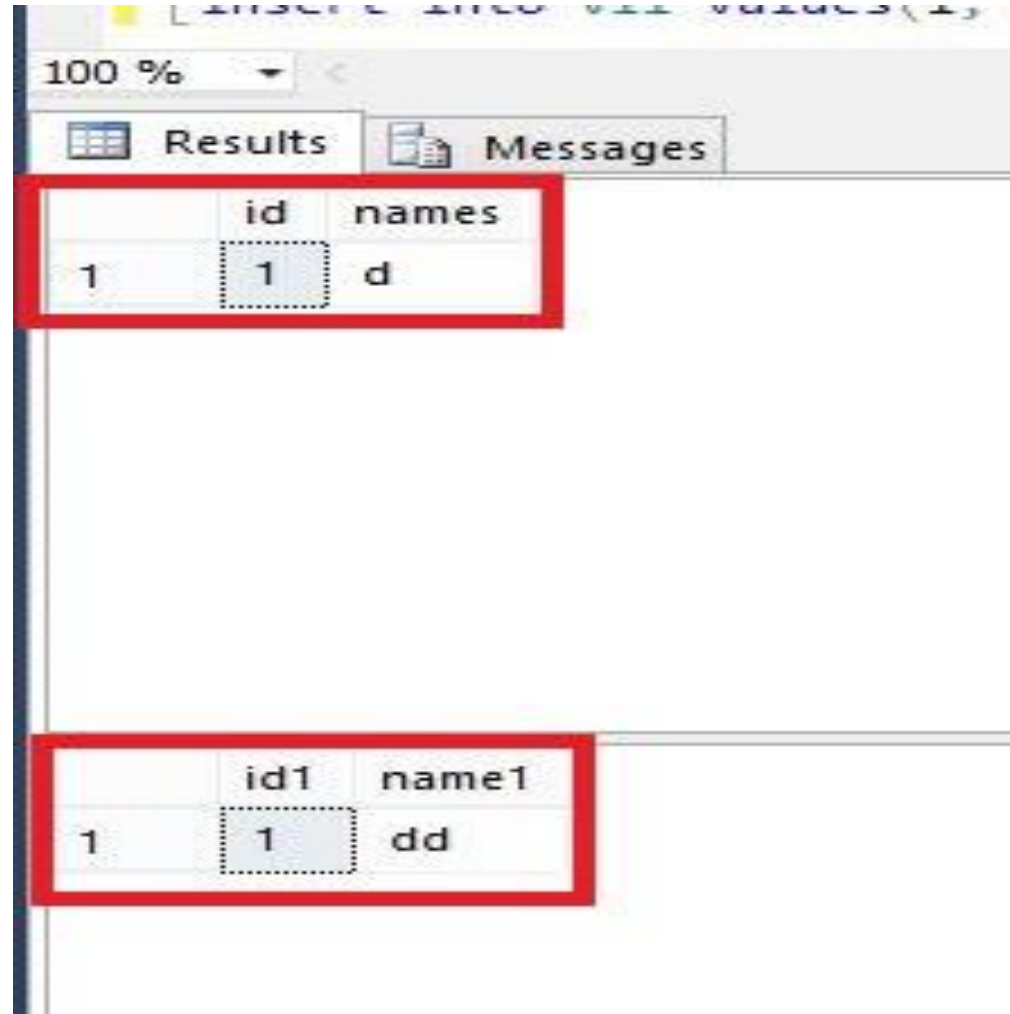
```
insert into v11 values(1,'d','dd')
```

You can see both tables by the folowing query:

```
select * from emp
```

```
select * from emp1values
```

DML Trigger(conti...)



100 %

Results Messages

	id	names
1	1	d

	id1	name1
1	1	dd

10. Procedures in SQL/PL SQL

PL/SQL is a block-structured language that enables developers to combine the power of SQL with procedural statements. A stored procedure in PL/SQL is nothing but a series of declarative SQL statements which can be stored in the database catalogue. A procedure can be thought of as a function or a method. They can be invoked through triggers, other procedures, or applications on Java, PHP etc. All the statements of a block are passed to Oracle engine all at once which increases processing speed and decreases the traffic.

10.1 Advantages of Procedures

Advantages:

- They result in performance improvement of the application. If a procedure is being called frequently in an application in a single connection, then the compiled version of the procedure is delivered.
- They reduce the traffic between the database and the application, since the lengthy statements are already fed into the database and need not be sent again and again via the application.
- They add to code reusability, similar to how functions and methods work in other languages such as C/C++ and Java.

10.2 Disadvantages of Procedures

Disadvantages:

- Stored procedures can cause a lot of memory usage. The database administrator should decide an upper bound as to how many stored procedures are feasible for a particular application.
- MySQL does not provide the functionality of debugging the stored procedures.

10.3 Syantax for Procedure

Syntax to create a procedure

```
CREATE PROCEDURE procedure_name  
AS  
sql_statement  
GO;
```

Execute a Stored Procedure

```
EXEC procedure_name;
```